

Propuesta de Proyecto: VoltGuard

Smart Breaker & Telemetría de Energía IoT

Programa EDSIA 2026 – Reto Final

3 de septiembre de 2026

Integrantes del Equipo

- Integrante 1 : Reyes Casanova Luis Khaled
- Integrante 2 : Juliana Hernández Hernández
- Integrante 3 : Jafet De Los Ángeles Palacios Guatzozón

Arquitectura del Sistema

- **Hardware & Embedded Systems:** Diseño e integración de circuito de CA, sensor de corriente, optocoplador y firmware en microcontrolador (ESP32 / Arduino).
 - **Backend & Software Architecture:** API REST en FastAPI, modelos SQLAlchemy 2.x, esquemas Pydantic V2, motor de reglas de corte y migraciones con Alembic.
 - **IA Inferencia & DevOps/CI-CD:** Módulo de predicción de consumo/costo, containerización en Docker, pipeline CI/CD en GitHub Actions ($\geq 80\%$ cobertura) y despliegue en producción.
-

1. Sentido del Proyecto y Problema Real

En entornos domésticos, laboratorios de electrónica e industrias pequeñas, el consumo de energía eléctrica suele ser una “caja negra”: los usuarios solo conocen su consumo cuando llega el recibo de luz. Además, las sobrecargas, cortos o sobretensiones en dispositivos específicos pueden causar daños irreparables en equipos sensibles sin que exista un mecanismo de protección proactivo.

VoltGuard resuelve este problema actuando como un **Smart Breaker IoT (Interrupidor Inteligente y PDU de Telemetría)**. El sistema no solo monitorea el voltaje, corriente RMS y consumo acumulado por dispositivo en tiempo real, sino que evalúa umbrales de seguridad en la nube para interrumpir automáticamente la corriente (corte por relé) ante una anomalía eléctrica grave, notificando al usuario en tiempo real.

2. Métrica del Proyecto y Alcance del MVP

Métrica de Valor (Lo que demuestra el éxito del producto)

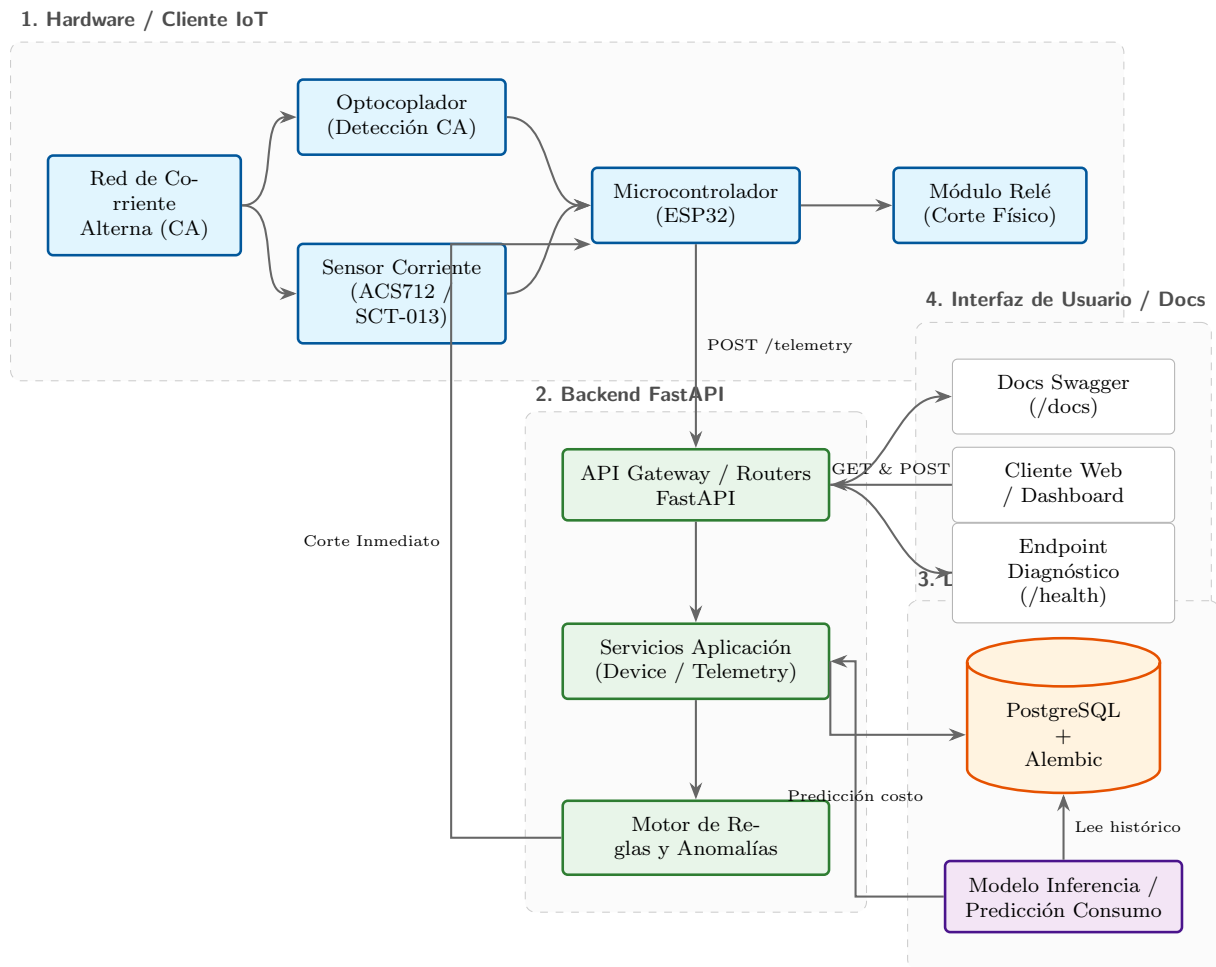
- **Tiempo de Reacción de Seguridad:** Tiempo < 500 ms desde que la API detecta una sobrecarga crítica hasta que emite la orden de apagado remoto (*breaker trigger*).
- **Precisión de Proyección:** Estimación del costo del recibo eléctrico mensual con un margen de error menor al 10 % basado en el histórico de lectura.
- **Disponibilidad y Calidad:** API viva en producción con 100 % de los tests en verde, ≥ 80 % de cobertura que bloquea el pipeline y cero errores en `ruff` y `mypy`.

Alcance Funcional (MVP)

1. **Ingesta de Telemetría (RF-1/RF-2):** Endpoint POST `/telemetry/readings` para recibir corriente RMS y voltaje calculados por el microcontrolador y asociarlos a un `device_id`.
2. **Motor de Reglas y Anomalías (RF-4):** Evaluación del consumo contra umbrales configurables por perfil. Si la corriente excede el máximo permitido, el backend emite un evento `CRITICAL_OVERLOAD` y envía la orden de apagado.
3. **Gestión de Dispositivos y Perfiles (RF-1):** CRUD de interruptores/dispositivos con capacidad de corte manual o automático.
4. **Consultas y Métricas de Consumo (RF-3/RF-6):** Agregaciones de consumo eléctrico por día/mes y cálculo del costo acumulado.
5. **Inferencia de IA de Apoyo:** Algoritmo en Python (Scikit-learn/Statsmodels) que proyecta la tendencia de consumo mensual y detecta comportamiento anómalo en aparatos (p. ej., un refrigerador consumiendo más de lo normal).

3. Diagrama de Bloques del Sistema

A continuación se presenta la arquitectura general del sistema corregida en TikZ:



4. Coincidencia con el “Piso Profesional” de la Competencia

- **Arquitectura en Capas:** Separación limpia entre Dominio, Aplicación, Infraestructura (SQLAlchemy) y Presentación (FastAPI).
- **Contenedores:** Dockerfile multi-stage ligero (python:3.12-slim) y docker-compose.yml que orquesta la API y PostgreSQL con volúmenes persistentes.
- **Calidad y Pipeline:** CI en GitHub Actions con ruff, mypy y pytest con restricción estricta de fail-under = 80.
- **Observabilidad:** Structured JSON Logging para eventos de sobrecarga y /health expuesto.